



**MEDS**

Maktab-e-Digital Systems  
Department of Electrical Engineering • UET Lahore

---

# MODULE 1

---

## Development Environment, Shell & Git

Summer Training Programme 2026 • Cohort 4

**Duration: 20 Hours**

Week 1 of 8-Week Intensive Training

Prepared by: Umer Shahid, Shehzeen Malik, & Aman Murad

Version 1.0 • April 2026



# Table of Contents

---

|                                                                  |    |
|------------------------------------------------------------------|----|
| Table of Contents.....                                           | 2  |
| 1. Module Overview.....                                          | 5  |
| 2. Learning Objectives.....                                      | 6  |
| 3. Weekly Schedule.....                                          | 7  |
| 4. Day 1 — Linux Fundamentals & Shell Navigation.....            | 8  |
| 4.1 The Terminal & Shell Basics.....                             | 8  |
| 4.2 File & Directory Operations.....                             | 9  |
| 4.3 File Permissions & Ownership.....                            | 9  |
| 4.4 Pipes & Redirection.....                                     | 10 |
| 4.5 Text Processing Tools.....                                   | 10 |
| 4.6 SSH Key Setup.....                                           | 11 |
| 4.7 Day 1 Exercises.....                                         | 11 |
| 5. Day 2 — Shell Scripting & Automation.....                     | 13 |
| 5.1 Your First Script.....                                       | 13 |
| 5.2 Variables & Data Types.....                                  | 13 |
| 5.3 Conditionals.....                                            | 14 |
| 5.4 Loops.....                                                   | 15 |
| 5.5 Command-Line Arguments.....                                  | 16 |
| 5.6 Functions.....                                               | 17 |
| 5.7 Error Handling & Best Practices.....                         | 17 |
| 5.8 Day 2 Exercises.....                                         | 18 |
| 6. Day 3 — Git Fundamentals & Version Control.....               | 19 |
| 6.1 Why Git?.....                                                | 19 |
| 6.2 The Git Data Model (Conceptual).....                         | 19 |
| 6.3 Initial Setup.....                                           | 19 |
| 6.4 Core Workflow: init → add → commit.....                      | 20 |
| 6.5 Inspecting History.....                                      | 20 |
| 6.6 .gitignore.....                                              | 21 |
| 6.7 Branching Basics.....                                        | 21 |
| 6.8 Working with GitHub (Remotes).....                           | 22 |
| 6.9 Day 3 Exercises.....                                         | 23 |
| 7. Day 4 — Advanced Git: Branching, Merging & Collaboration..... | 24 |
| 7.1 Merging Branches.....                                        | 24 |
| 7.2 Merge Conflicts — The Most Important Skill.....              | 24 |
| 7.3 Rebasing.....                                                | 25 |
| 7.4 Git Stash.....                                               | 26 |
| 7.5 Cherry-Pick.....                                             | 26 |



|                                                              |    |
|--------------------------------------------------------------|----|
| 7.6 The Pull Request Workflow.....                           | 27 |
| 7.7 Undoing Mistakes.....                                    | 27 |
| 7.8 Day 4 Exercises.....                                     | 28 |
| 8. Day 5 — Build Systems (Makefiles) & Grand Assignment..... | 29 |
| 8.1 Why Makefiles?.....                                      | 29 |
| 8.2 Makefile Basics.....                                     | 29 |
| 8.3 Variables & Automatic Variables.....                     | 29 |
| 8.4 Phony Targets & Organization.....                        | 30 |
| 8.5 A Real-World Makefile (Multi-file C Project).....        | 30 |
| 8.6 Day 5 Exercises.....                                     | 31 |
| 9. Grand Assignment: Module 1 Capstone.....                  | 33 |
| 9.1 Project Description.....                                 | 33 |
| 9.2 Repository Structure.....                                | 33 |
| 9.3 Requirements.....                                        | 33 |
| 9.4 Sample Log Format.....                                   | 35 |
| 9.5 Expected Output Example.....                             | 35 |
| 9.6 Bonus Challenges (+10 points).....                       | 36 |
| 10. Assessment Rubric.....                                   | 37 |
| 11. Recommended Resources.....                               | 38 |
| Primary References.....                                      | 38 |
| Video Resources.....                                         | 38 |
| Practice Platforms.....                                      | 38 |
| Quick References.....                                        | 38 |
| 12. Appendix: Quick Reference Cheat Sheets.....              | 39 |
| A. Essential Linux Commands.....                             | 39 |
| B. Git Commands Quick Reference.....                         | 39 |
| C. Makefile Quick Reference.....                             | 40 |



# 1. Module Overview

---

Module 1 is the gateway to your MEDS Lab journey. Before writing a single line of Verilog or SystemVerilog, you need to master the tools that professional hardware engineers use daily. This module covers the Linux command line, shell scripting, Git version control, and Makefiles — the foundational toolkit that underpins everything you will do at MEDS for the next two years.

This module is inspired by MIT’s “The Missing Semester of Your CS Education” — a course that recognizes that universities often skip the practical tools that engineers spend most of their time using. At MEDS, we fix that gap in Week 1.

## Why This Matters for Hardware Engineers

EDA tools (Verilator, VCS, Vivado) are command-line driven. RISC-V test suites are built with Make. Open-source contributions happen through Git and GitHub. If you cannot navigate a terminal confidently, you cannot contribute to a real project.

**Format:** 4 hours/day for 5 days = 20 hours total.

**Pre-requisites:** A laptop with Ubuntu 22.04+ (native or WSL2 on Windows). No prior Linux experience required.

**Deliverable:** A Grand Assignment submitted as a Git repository on GitHub, demonstrating mastery of all topics.



## 2. Learning Objectives

---

By the end of this module, every mentee should be able to:

### Linux & Shell

- Navigate the Linux filesystem confidently using `cd`, `ls`, `find`, and `tree`
- Understand file permissions (`rw`x), ownership, and use `chmod`/`chown` correctly
- Use pipes (`|`), redirection (`>`, `>>`, `<`, `2>&1`), and process substitution effectively
- Manage processes with `ps`, `top`, `kill`, `jobs`, `bg`, `fg`, and understand signals
- Use essential text-processing tools: `grep`, `sed`, `awk`, `sort`, `uniq`, `cut`, `wc`, `head`, `tail`
- Create and use SSH keys for secure remote access and GitHub authentication

### Shell Scripting (Bash)

- Write well-structured Bash scripts with proper shebang, comments, and exit codes
- Use variables, conditionals (`if/elif/else`), loops (`for`, `while`), and case statements
- Handle command-line arguments using `$1`, `$#`, `$@`, and `getopts`
- Write functions, handle errors with `set -euo pipefail`, and use `trap` for cleanup
- Automate repetitive tasks like file renaming, log parsing, and build orchestration

### Git Version Control

- Initialize repositories, stage changes, and create meaningful commits
- Understand the Git object model (blobs, trees, commits, refs) conceptually
- Create, switch, and manage branches; understand HEAD and detached HEAD
- Merge branches and resolve merge conflicts manually and confidently
- Use interactive rebase, cherry-pick, and stash for workflow management
- Work with remotes: clone, fetch, pull, push, and track upstream branches
- Write proper `.gitignore` files and understand why certain files must not be tracked
- Use GitHub for collaboration: pull requests, code review, and issue tracking

### Build Systems (Make)

- Write Makefiles with targets, prerequisites, and recipes
- Understand and use automatic variables (`$@`, `$<`, `$^`) and pattern rules
- Create phony targets (`clean`, `all`, `test`) and organize multi-file build systems
- Write a Makefile that compiles a multi-file C project and runs tests



### 3. Weekly Schedule

The 20-hour module is spread across 5 working days. Each day follows the same structure: 2 hours of guided instruction followed by 2 hours of hands-on exercises.

| Day          | Topics                                                                                                                   | Hours | Mode |
|--------------|--------------------------------------------------------------------------------------------------------------------------|-------|------|
| <b>Day 1</b> | Linux Fundamentals & Shell Navigation: Filesystem, permissions, pipes, redirection, text processing, SSH keys            | 4h    | Lab  |
| <b>Day 2</b> | Shell Scripting & Automation: Bash scripts, variables, loops, conditionals, command-line args, functions, error handling | 4h    | Lab  |
| <b>Day 3</b> | Git Fundamentals: Init, add, commit, log, diff, branching basics, .gitignore, GitHub setup                               | 4h    | Lab  |
| <b>Day 4</b> | Advanced Git: Merge conflicts, rebasing, stash, cherry-pick, pull requests, code review workflow                         | 4h    | Lab  |
| <b>Day 5</b> | Makefiles & Grand Assignment: Make targets, variables, pattern rules; Grand Assignment briefing & start                  | 4h    | Lab  |

**Total Hours:** 20 hours

**Grand Assignment Completion:** Due 48 hours after Day 5 session (submitted as GitHub repo)



## 4. Day 1 — Linux Fundamentals & Shell Navigation

**Duration:** 4 hours

**Goal:** Navigate the Linux filesystem, manage files/permissions, and use pipes and text processing tools with confidence.

### 4.1 The Terminal & Shell Basics

A shell is a program that takes commands from the keyboard and gives them to the operating system to execute. Bash (Bourne Again SHell) is the default shell on most Linux distributions and the one we will use throughout MEDS.

When you open a terminal, you see a prompt that typically looks like:

```
username@hostname:~$
```

This tells you: who you are (username), which machine (hostname), and where you are in the filesystem (~, your home directory). The \$ indicates a regular user; # indicates root.

### Essential Navigation Commands

|                              |                                                  |
|------------------------------|--------------------------------------------------|
| <code>pwd</code>             | # Print Working Directory — where am I?          |
| <code>ls</code>              | # List contents of current directory             |
| <code>ls -la</code>          | # Long format, including hidden files (dotfiles) |
| <code>ls -lh</code>          | # Human-readable file sizes                      |
| <code>cd /path/to/dir</code> | # Change directory (absolute path)               |
| <code>cd ../</code>          | # Go up one level (relative path)                |
| <code>cd ~</code>            | # Go to home directory                           |
| <code>cd -</code>            | # Go to previous directory (toggle)              |
| <code>tree -L 2</code>       | # Show directory tree, 2 levels deep             |
| <code>file myfile</code>     | # Determine file type                            |

#### Pro Tip: Tab Completion

Press TAB to auto-complete file and directory names. Press TAB twice to see all possible completions. This will save you hundreds of keystrokes every day.

### Absolute vs. Relative Paths

An absolute path starts from the root directory (/). A relative path starts from your current directory. Understanding the difference is critical:

|                                             |                                     |
|---------------------------------------------|-------------------------------------|
| <code>/home/umer/projects/riscv-test</code> | # Absolute: always works            |
| <code>./src/main.c</code>                   | # Relative: from current dir        |
| <code>../include/header.h</code>            | # Relative: one level up, then into |
| <code>include/</code>                       |                                     |



```
~/projects # ~ expands to your home directory
```

## 4.2 File & Directory Operations

```
mkdir -p project/src/rtl # Create nested directories
touch file.txt # Create empty file
cp file.txt backup.txt # Copy file
cp -r dir1/ dir2/ # Copy directory recursively
mv old_name.txt new_name.txt # Rename/move file
rm file.txt # Delete file (PERMANENT - no trash!)
rm -rf directory/ # Delete directory recursively
(DANGEROUS)
ln -s /path/to/target link_name # Create symbolic link
```

### Warning: rm is permanent!

Linux has no recycle bin. When you rm a file, it is gone forever. Always double-check before using rm -rf. A common disaster: rm -rf / (deletes EVERYTHING). Never run commands you do not understand.

## 4.3 File Permissions & Ownership

Linux file permissions control who can read (r), write (w), and execute (x) files. Every file has three permission groups: owner, group, and others.

```
ls -la
# -rw-r--r-- 1 user user 1234 Apr 27 10:00 file.txt
# drwxr-xr-x 2 user user 4096 Apr 27 10:00 directory/

# Permission string breakdown:
# d rwx r-x r-x
# | | | |
# | | | +-- Others: read + execute
# | | +----- Group: read + execute
# | +----- Owner: read + write + execute
# +----- d = directory, - = file, l = link
```

Numeric (octal) permissions: r=4, w=2, x=1. Add them up for each group:

```
chmod 755 script.sh # rwxr-xr-x (owner: all, group/others: read+exec)
chmod 644 file.txt # rw-r--r-- (owner: read+write, others: read only)
chmod +x script.sh # Add execute permission for all
chown user:staff file # Change owner and group
```





## 4.4 Pipes & Redirection

Pipes and redirection are the heart of the Unix philosophy: small programs that do one thing well, connected together to solve complex problems.

### Redirection

```
echo 'hello' > file.txt           # Write to file (overwrite)
echo 'world' >> file.txt          # Append to file
cat < file.txt                    # Read from file as stdin
command 2> errors.log             # Redirect stderr only
command > out.log 2>&1            # Redirect both stdout and stderr
command &> all.log                 # Same as above (Bash shorthand)
```

### Pipes

The pipe operator (|) sends the output of one command as input to the next:

```
ls -la | grep '.sv'              # Find SystemVerilog files
cat log.txt | sort | uniq -c | sort -rn # Count unique lines, sorted
history | grep 'git' | tail -20   # Last 20 git commands you ran
find . -name '*.v' | wc -l        # Count all Verilog files
```

## 4.5 Text Processing Tools

These tools are your bread and butter for parsing logs, processing data, and automating tasks:

### grep — Search for patterns

```
grep 'error' simulation.log      # Find lines containing 'error'
grep -i 'warning' sim.log        # Case-insensitive search
grep -r 'TODO' ./src/            # Recursive search in directory
grep -n 'assign' design.sv       # Show line numbers
grep -c 'PASS' test_results.log  # Count matching lines
grep -v 'debug' sim.log          # Invert: lines NOT matching
grep -E 'error|warning|fatal' sim.log # Extended regex: OR pattern
```

### sed — Stream Editor

```
sed 's/old/new/' file.txt        # Replace first occurrence per line
sed 's/old/new/g' file.txt       # Replace ALL occurrences (global)
sed -i 's/old/new/g' file.txt    # In-place edit (modifies file!)
sed -n '10,20p' file.txt         # Print lines 10-20
sed '/^#/d' config.txt           # Delete comment lines
```



## awk — Pattern Processing

```
awk '{print $1}' file.txt           # Print first column
awk -F',' '{print $2}' data.csv     # CSV: print 2nd field
awk '/error/ {count++} END {print count}' log.txt # Count error lines
awk '{sum+=$1} END {print sum}' numbers.txt      # Sum a column
```

## Other Essential Tools

```
sort file.txt                     # Sort lines alphabetically
sort -n numbers.txt              # Sort numerically
uniq                             # Remove adjacent duplicate lines
uniq -c                          # Count occurrences of each line
cut -d',' -f2 data.csv           # Extract 2nd field (comma delimiter)
wc -l file.txt                   # Count lines
head -20 file.txt                # First 20 lines
tail -f simulation.log           # Follow file in real-time (live logs)
diff file1.txt file2.txt         # Compare two files
which verilator                  # Find location of a command
man grep                         # Read the manual for grep
```

## 4.6 SSH Key Setup

SSH keys are essential for secure, password-free authentication to GitHub and remote servers. You will set this up today and use it for the rest of your time at MEDS.

```
ssh-keygen -t ed25519 -C 'your_email@example.com'
# Press Enter for default location (~/.ssh/id_ed25519)
# Enter a passphrase (recommended) or leave blank

eval '$(ssh-agent -s)'          # Start SSH agent
ssh-add ~/.ssh/id_ed25519       # Add your key

cat ~/.ssh/id_ed25519.pub       # Copy this public key to GitHub
# Go to GitHub > Settings > SSH Keys > New SSH Key > paste

ssh -T git@github.com           # Test connection
```

## 4.7 Day 1 Exercises

### Day 1 Hands-On Exercises (2 hours)



- ▶ Exercise 1: Create the following directory structure using only mkdir and touch: project/{src/{rtl,tb,include},docs,scripts,build}. Verify with tree.
- ▶ Exercise 2: Download the RISC-V ISA manual PDF using wget/curl. Find its size, permissions, and file type using ls -lh and file.
- ▶ Exercise 3: Create a file with 100 random lines using a command. Sort it, find unique lines, and count occurrences. Pipe the top 10 most frequent to a new file.
- ▶ Exercise 4: Set up SSH keys and verify connection to GitHub using ssh -T git@github.com. Take a screenshot of the success message.
- ▶ Exercise 6 (Bonus): Write a one-liner pipeline that finds all .c files in /usr/include, counts lines in each, sorts by line count, and shows the top 5 largest files.



## 5. Day 2 — Shell Scripting & Automation

**Duration:** 4 hours

**Goal:** Write robust Bash scripts that automate tasks, handle arguments, and manage errors gracefully.

### 5.1 Your First Script

A shell script is simply a text file containing commands that the shell executes in sequence. The first line (shebang) tells the OS which interpreter to use:

```
#!/bin/bash
# My first MEDS script
# Author: Your Name
# Date: 2026-05-XX

echo "Hello, MEDS Lab!"
echo "Today is $(date)"
echo "You are logged in as: $(whoami)"
echo "Your home directory is: $HOME"
```

To run it:

```
chmod +x hello.sh      # Make it executable
./hello.sh             # Run it
```

#### Always Use #!/bin/bash

The shebang line (#!/bin/bash) is not just a comment. It tells the kernel which interpreter to use. Without it, the script may run under a different shell (sh, dash) which has different syntax rules. Always include it.

### 5.2 Variables & Data Types

Bash variables are untyped — everything is a string unless used in an arithmetic context:

```
# Variable assignment (NO spaces around =)
NAME="MEDS Lab"
COUNT=42
FILES=$(ls *.sv 2>/dev/null)    # Command substitution
TIMESTAMP=$(date +%Y%m%d_%H%M%S)

# Using variables (always quote to handle spaces)
echo "Welcome to $NAME"
```



```
echo "File count: ${COUNT}"
echo "Current dir: $(pwd)"

# Arithmetic
A=10
B=3
SUM=$((A + B))           # Integer arithmetic only
echo $((A * B))           # 30
echo $((A / B))           # 3 (integer division)
echo $((A % B))           # 1 (modulo)
```

### Common Mistake: Spaces Around =

NAME = "value" is WRONG and will cause an error. Bash interprets NAME as a command. Always write NAME="value" with no spaces around the equals sign.

## 5.3 Conditionals

```
#!/bin/bash

# File existence checks
if [ -f "design.sv" ]; then
    echo "Design file found"
elif [ -d "src/" ]; then
    echo "Source directory exists but no design file"
else
    echo "Nothing found – creating project structure"
    mkdir -p src/
fi

# String comparison
if [ "$USER" = "root" ]; then
    echo "Running as root – be careful!"
fi

# Numeric comparison
if [ "$COUNT" -gt 100 ]; then
    echo "More than 100 files"
fi
```



```
# Common test operators:
# -f file      File exists and is regular file
# -d dir       Directory exists
# -z str       String is empty
# -n str       String is non-empty
# -eq -ne -lt -le -gt -ge  Numeric comparisons
# = !=        String comparisons
```

## 5.4 Loops

### For Loops

```
# Iterate over files
for file in *.sv; do
    echo "Processing: $file"
    wc -l "$file"
done

# C-style for loop
for ((i=0; i<10; i++)); do
    echo "Iteration $i"
done

# Iterate over a range
for i in {1..5}; do
    echo "Test case $i"
done

# Iterate over command output
for dir in $(find . -type d -name "tb_*"); do
    echo "Testbench directory: $dir"
done
```

### While Loops

```
# Read file line by line (correct way)
while IFS= read -r line; do
    echo "Line: $line"
done < input.txt

# Counter-based while loop
COUNT=0
```



```
while [ $COUNT -lt 5 ]; do
    echo "Count: $COUNT"
    COUNT=$((COUNT + 1))
done
```

## 5.5 Command-Line Arguments

Scripts become powerful tools when they accept arguments:

```
#!/bin/bash
# Usage: ./build.sh <module_name> [--verbose]

# Special variables:
# $0      Script name
# $1      First argument
# $2      Second argument
# $#      Number of arguments
# $@      All arguments (as separate words)
# $*      All arguments (as single string)
# $?      Exit code of last command

# Example: A simple build script
if [ $# -lt 1 ]; then
    echo "Usage: $0 <module_name> [--verbose]"
    exit 1
fi

MODULE=$1
VERBOSE=0

# Parse optional flags
for arg in "$@"; do
    case "$arg" in
        --verbose) VERBOSE=1 ;;
        --clean)   rm -rf build/ ;;
    esac
done

echo "Building module: $MODULE"
if [ $VERBOSE -eq 1 ]; then
    echo "Verbose mode enabled"
```



```
fi
```

## 5.6 Functions

```
#!/bin/bash

# Function definition
compile_module() {
    local module_name="$1"    # local scope
    local output_dir="$2"

    if [ -z "$module_name" ]; then
        echo "Error: Module name required"
        return 1              # Return non-zero on error
    fi

    echo "Compiling: $module_name"
    mkdir -p "$output_dir"
    # ... compilation commands ...
    return 0
}

# Function call
compile_module "alu" "build/"
if [ $? -eq 0 ]; then
    echo "Compilation successful"
fi
```

## 5.7 Error Handling & Best Practices

```
#!/bin/bash

set -euo pipefail    # The holy trinity of Bash safety

# set -e      Exit immediately if any command fails
# set -u      Error on undefined variables
# set -o pipefail  Catch errors in pipes (not just last command)

# Trap for cleanup on exit
cleanup() {
    echo "Cleaning up temporary files..."
    rm -f /tmp/meds_build_*
```





```
}  
trap cleanup EXIT      # Run cleanup() on any exit (normal or error)  
  
# Logging function  
log() {  
    echo "[$(date +%H:%M:%S)] $*"   
}  
  
log "Starting build process"  
# ... rest of script ...
```

## 5.8 Day 2 Exercises

### Day 2 Hands-On Exercises

- ▶ Exercise 1: Write a script `organizer.sh` that takes a directory path as an argument and organizes files into subdirectories by extension (`.sv` → `verilog/`, `.c` → `c_code/`, `.txt` → `docs/`, etc.).
- ▶ Exercise 2: Write a script `file_stats.sh` that takes a directory as input and prints: total number of files, total number of directories, largest file (name + size), most recently modified file.
- ▶ Exercise 3: Write a script `sim_checker.sh` that reads a simulation log file, counts `ERRORs`, `WARNINGs`, and `PASSEd` tests, and prints a summary. Exit with code 1 if any errors found.
- ▶ Exercise 4: Write a `batch_rename.sh` script that takes a prefix, a suffix, and a directory. It renames all files in that directory from `<prefix>_old_<N>.sv` to `<suffix>_new_<N>.sv`. Use proper error handling.



## 6. Day 3 — Git Fundamentals & Version Control

**Duration:** 4 hours

**Goal:** Understand the Git data model, make meaningful commits, and set up a professional GitHub workflow.

### 6.1 Why Git?

Version control is not optional in professional engineering. Without Git, you face these problems: you cannot undo mistakes, you cannot work on multiple features simultaneously, you cannot collaborate without overwriting each other's work, and you have no history of what changed, when, and why.

At MEDS, every project, every test suite contribution, and every piece of RTL lives in a Git repository. Your GitHub profile is your engineering CV. Treat it accordingly.

### 6.2 The Git Data Model (Conceptual)

Understanding Git's internal model makes everything else click. Git stores data as a directed acyclic graph (DAG) of snapshots:

- Blob: A file's content (just the bytes, no filename)
- Tree: A directory listing — maps filenames to blobs or other trees
- Commit: A snapshot = a tree + metadata (author, date, message, parent commit)
- Reference (ref): A human-readable name pointing to a commit (e.g., main, HEAD)

Every commit points to its parent(s), forming a chain. A branch is just a pointer to a commit. HEAD is a pointer to the current branch (or commit if detached).

### 6.3 Initial Setup

```
# Configure your identity (do this once)
git config --global user.name "Your Full Name"
git config --global user.email "your_email@example.com"

# Useful defaults
git config --global init.defaultBranch main
git config --global core.editor 'vim'           # or nano, code, etc.
git config --global pull.rebase false           # Default merge strategy
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.lg 'log --oneline --graph --all --decorate'
```



```
# Verify your config
git config --list
```

## 6.4 Core Workflow: init → add → commit

```
# Create a new repository
mkdir riscv-alu && cd riscv-alu
git init

# Create some files
echo "# RISC-V ALU" > README.md
mkdir -p src/ tb/
touch src/alu.sv tb/tb_alu.sv

# Check status — do this constantly!
git status

# Stage files (add to staging area / index)
git add README.md
git add src/ tb/          # Add entire directories
git add .                 # Add everything (use cautiously)
git add -p                # Interactive: review each change before staging

# Commit with a meaningful message
git commit -m "Initial project structure with ALU module and testbench stubs"
```

### Writing Good Commit Messages

Use imperative mood: "Add ALU module" not "Added ALU module". First line: 50 chars max, summary. Blank line then detailed body if needed. Reference issue numbers: "Fix sign extension bug (#42)". Bad examples: "fixed stuff", "wip", "asdfg". Your commit history tells the story of your project — make it readable.

## 6.5 Inspecting History

```
git log                # Full commit log
git log --oneline       # Compact view
git log --oneline --graph --all  # Visual branch graph
git log --author='Umer'    # Filter by author
git log -p               # Show diffs with each commit
```



```
git log --stat                # Show file change summary

git diff                      # Unstaged changes vs. last commit
git diff --staged             # Staged changes vs. last commit
git diff HEAD~3 HEAD          # Compare with 3 commits ago
git diff branch1..branch2     # Compare two branches

git show abc1234              # Show a specific commit
git blame src/alu.sv          # Who changed each line?
```

## 6.6 .gitignore

Some files should NEVER be committed: build artifacts, simulation waveforms, editor config, OS files. Create a .gitignore file at the repository root:

```
# Build artifacts
build/
*.o
*.d

# Simulation outputs
*.vcd
*.fsdb
*.vpd
simv
simv.daidir/
csrc/

# Editor and OS files
.vscode/
*.swp
.DS_Store
Thumbs.db

# Tool-generated
*.log
!important.log      # Exception: track this specific log
```

## 6.7 Branching Basics

```
# Create and switch to a new branch
git branch feature/add-multiplier  # Create branch
```



```
git checkout feature/add-multiplier    # Switch to it
git checkout -b feature/add-multiplier # Create + switch (shorthand)
git switch -c feature/add-multiplier   # Modern syntax (Git 2.23+)

# List branches
git branch                             # Local branches
git branch -a                          # All branches (including remote)
git branch -v                          # Branches with last commit

# Switch branches
git checkout main                      # Switch to main
git switch main                       # Modern syntax

# Delete a branch (after merging)
git branch -d feature/add-multiplier   # Safe delete
git branch -D feature/add-multiplier   # Force delete
```

## Branching Naming Convention at MEDS

- feature/<description> — New feature (e.g., feature/add-mul-unit)
- fix/<description> — Bug fix (e.g., fix/sign-extension-rv64)
- test/<description> — Adding tests (e.g., test/alu-overflow-cases)
- docs/<description> — Documentation changes

## 6.8 Working with GitHub (Remotes)

```
# Clone an existing repository
git clone git@github.com:meds-uet/riscv-alu.git

# Add a remote to an existing local repo
git remote add origin git@github.com:yourusername/riscv-alu.git
git remote -v                                # List remotes

# Push your branch to GitHub
git push -u origin main                      # First push (sets upstream)
git push                                    # Subsequent pushes

# Fetch and pull updates
git fetch origin                            # Download changes (don't merge)
git pull origin main                        # Fetch + merge
```



## 6.9 Day 3 Exercises

### Day 3 Hands-On Exercises

- ▶ Exercise 1: Initialize a local repo called medss-module1. Create a README.md with your name and student ID. Make 5 meaningful commits, each adding a different file (e.g., notes from each day). Use `git log --oneline --graph` to visualize.
- ▶ Exercise 2: Create a proper .gitignore for a hardware project (Verilog/SV). Test it by creating files that should and should not be ignored. Verify with `git status`.
- ▶ Exercise 3: Create a repository on GitHub. Push your local repo to GitHub using SSH. Verify by visiting the GitHub page in your browser.
- ▶ Exercise 4: Practice `git diff`: modify a file, view the diff unstaged, stage it, view the staged diff. Undo the staging with `git restore --staged`.
- ▶ Exercise 5: Create two branches from main. Make different commits on each branch. Use `git log --all --graph --oneline` to visualize the divergence.
- ▶ Exercise 6: Use `git blame` on a file from a public repo (e.g., riscv-arch-test). Identify who wrote a specific line and when.



## 7. Day 4 — Advanced Git: Branching, Merging & Collaboration

**Duration:** 4 hours (2h guided + 2h hands-on)

**Goal:** Resolve merge conflicts, use rebase and stash, and practice the Pull Request workflow used in real open-source projects.

### 7.1 Merging Branches

When you have finished a feature branch, you merge it back into main. Git has two strategies:

#### Fast-Forward Merge

If main has not diverged (no new commits since branch was created), Git simply moves the main pointer forward:

```
git checkout main
git merge feature/add-shifter
# Fast-forward (no merge commit created)
```

#### Three-Way Merge

If main has new commits, Git creates a merge commit that has two parents:

```
git checkout main
git merge feature/add-alu-flags
# Creates a merge commit
```

### 7.2 Merge Conflicts — The Most Important Skill

Merge conflicts happen when two branches modify the same line(s) of the same file. This is normal and expected in team projects. Do not fear conflicts — learn to resolve them.

#### Creating a Conflict (Hands-On Demo)

```
# Setup: create a conflict scenario
git checkout -b branch-a
# Edit line 5 of alu.sv: assign result = a + b;
git commit -am "branch-a: add operation"

git checkout main
git checkout -b branch-b
# Edit same line 5 of alu.sv: assign result = a - b;
git commit -am "branch-b: subtract operation"
```



```
# Now merge branch-a into main
git checkout main
git merge branch-a    # Clean merge

# Now merge branch-b into main - CONFLICT!
git merge branch-b
# CONFLICT (content): Merge conflict in src/alu.sv
```

## What a Conflict Looks Like

```
<<<<<<< HEAD
assign result = a + b;    // From branch-a (already merged)
=====
assign result = a - b;    // From branch-b (being merged)
>>>>>>> branch-b
```

## Resolving the Conflict

Open the file in your editor. You have three choices:

1. Keep the HEAD version (branch-a's change)
2. Keep the incoming version (branch-b's change)
3. Write a new version that combines both (most common in real projects)

For example, the correct resolution might be:

```
// Resolved: support both operations with an opcode select
assign result = (opcode == ADD) ? a + b : a - b;
```

After editing:

```
git add src/alu.sv    # Mark as resolved
git commit            # Complete the merge (editor opens)
# Or: git merge --abort    # Cancel the merge entirely
```

### Never Leave Conflict Markers in Code!

The <<<<<<<, =====, and >>>>>>> markers must be completely removed. If you commit with these markers still in the file, the code is broken. Always check git diff before committing a merge resolution.

## 7.3 Rebasing

Rebase replays your branch's commits on top of the latest main, creating a linear history:

```
# Instead of merge, rebase your feature branch onto main
git checkout feature/add-decoder
```





```
git rebase main

# If conflicts occur during rebase:
# 1. Resolve the conflict in the file
# 2. git add <resolved-files>
# 3. git rebase --continue
# Or: git rebase --abort to cancel

# Interactive rebase: rewrite last N commits
git rebase -i HEAD~3
# Options: pick, reword, edit, squash, fixup, drop
```

### When to Rebase vs. Merge

Use rebase to clean up your local branch before pushing. Use merge when integrating into shared branches (main, develop). Golden rule: NEVER rebase commits that have been pushed to a shared remote branch.

## 7.4 Git Stash

Stash temporarily saves uncommitted changes so you can switch context:

```
# Save current changes
git stash                                # Stash tracked files
git stash -m "WIP: decoder logic"      # With descriptive message
git stash -u                            # Include untracked files

# List stashes
git stash list

# Restore stashed changes
git stash pop                           # Apply + remove from stash
git stash apply                         # Apply but keep in stash
git stash apply stash@{2}               # Apply specific stash

# Discard a stash
git stash drop stash@{0}
git stash clear                         # Remove all stashes
```

## 7.5 Cherry-Pick

Apply a specific commit from another branch without merging the entire branch:



```
git cherry-pick abc1234          # Apply commit abc1234 to current branch
git cherry-pick abc1234 def5678 # Apply multiple commits
```

## 7.6 The Pull Request Workflow

This is the workflow you will use for every MEDS project and every open-source contribution:

4. Create a feature branch from main
5. Make commits on your feature branch
6. Push your branch to GitHub
7. Open a Pull Request (PR) on GitHub
8. Request code review from your mentor
9. Address review feedback with new commits
10. Merge the PR once approved
11. Delete the feature branch

```
# The workflow in commands:
git checkout -b feature/add-register-file
# ... make changes, commit ...
git push -u origin feature/add-register-file
# Go to GitHub → Open Pull Request → Select reviewers
# After approval:
git checkout main
git pull origin main
git branch -d feature/add-register-file
```

## 7.7 Undoing Mistakes

```
# Undo unstaged changes to a file
git checkout -- file.sv          # Discard changes
git restore file.sv             # Modern syntax

# Unstage a file (keep changes)
git reset HEAD file.sv
git restore --staged file.sv     # Modern syntax

# Amend the last commit (message or content)
git commit --amend -m "Better message"

# Undo last commit (keep changes staged)
git reset --soft HEAD~1
```



```
# Undo last commit (keep changes unstaged)
git reset HEAD~1

# Undo last commit (DISCARD changes – dangerous!)
git reset --hard HEAD~1

# Create a new commit that reverses a previous commit (safe)
git revert abc1234
```

## 7.8 Day 4 Exercises

### Day 4 Hands-On Exercises

- ▶ Exercise 1 (Merge Conflict): Create a repo with a file. Create two branches that edit the same line. Merge one into main, then merge the other. Resolve the conflict. Show the resolution commit.
- ▶ Exercise 2 (Multi-file Conflict): Create a scenario where 3 files have conflicts. Resolve all three, verifying each with `git diff` before staging.
- ▶ Exercise 3 (Rebase): Create a feature branch with 3 commits. Add 2 commits to main. Rebase the feature branch onto main. Verify with `git log --graph`.
- ▶ Exercise 4 (Interactive Rebase): Make 5 small commits. Use `git rebase -i HEAD~5` to squash them into 2 meaningful commits. Verify the history.
- ▶ Exercise 5 (Pull Request): Fork any public repo on GitHub. Create a branch, make a change (e.g., fix a typo in README), push, and open a Pull Request. (You can close it after.)
- ▶ Exercise 6 (Stash): Make changes to two files. Stash them. Switch branches. Pop the stash on the new branch. Verify the changes applied correctly.



## 8. Day 5 — Build Systems (Makefiles) & Grand Assignment

**Duration:** 4 hours (2h guided + 2h hands-on)

**Goal:** Write Makefiles to automate builds, and receive the Grand Assignment that ties everything together.

### 8.1 Why Makefiles?

Every serious hardware project uses Make (or a similar build system). EDA tools like Verilator, VCS, and Vivado are invoked through Makefiles. The RISC-V architecture test suite uses Make. Learning Make now means you can contribute to any of these projects immediately.

### 8.2 Makefile Basics

A Makefile consists of rules. Each rule has a target, prerequisites, and a recipe:

```
# Basic structure:
target: prerequisites
recipe      # MUST use TAB, not spaces!

# Example: compile a C program
hello: hello.c
gcc -o hello hello.c

# Run it:
# make hello      (builds hello)
# make            (builds first target by default)
```

#### TABs, Not Spaces!

Makefile recipes MUST be indented with a TAB character, not spaces. This is the #1 source of Makefile errors. Configure your editor to show whitespace characters to catch this.

### 8.3 Variables & Automatic Variables

```
# Variables
CC = gcc
CFLAGS = -Wall -Wextra -g
SRC = main.c utils.c alu.c
OBJ = $(SRC:.c=.o)      # Pattern substitution: .c -> .o
TARGET = simulator
```



```
# Rule using variables
$(TARGET): $(OBJ)
$(CC) $(CFLAGS) -o $@ $^

# Pattern rule: how to build any .o from .c
%.o: %.c
$(CC) $(CFLAGS) -c $< -o $@

# Automatic variables:
# $@   Target name (e.g., simulator)
# $<   First prerequisite (e.g., main.c)
# $^   All prerequisites (e.g., main.o utils.o alu.o)
# $*   Stem of pattern match
```

## 8.4 Phony Targets & Organization

```
.PHONY: all clean test run

all: $(TARGET)      # Default target

clean:
rm -rf $(OBJ) $(TARGET) build/

test: $(TARGET)
./$(TARGET) --test-mode
@echo 'All tests passed!'    # @ suppresses command echo

run: $(TARGET)
./$(TARGET)

# Useful pattern: print any variable
print-%:
@echo '$* = $(*)'
# Usage: make print-SRC -> SRC = main.c utils.c alu.c
```

## 8.5 A Real-World Makefile (Multi-file C Project)

```
# Project: Simple RISC-V Instruction Decoder Simulator
CC      = gcc
CFLAGS  = -Wall -Wextra -std=c11 -g
```



```
LDFLAGS =

SRC_DIR = src
BUILD_DIR = build
BIN_DIR = bin

SRC      = $(wildcard $(SRC_DIR)/*.c)
OBJ      = $(SRC:$(SRC_DIR)/%.c=$(BUILD_DIR)/%.o)
TARGET   = $(BIN_DIR)/decoder

.PHONY: all clean test dirs

all: dirs $(TARGET)

dirs:
@mkdir -p $(BUILD_DIR) $(BIN_DIR)

$(TARGET): $(OBJ)
$(CC) $(LDFLAGS) -o $@ $^
@echo 'Build successful: $@'

$(BUILD_DIR)/%.o: $(SRC_DIR)/%.c
$(CC) $(CFLAGS) -c $< -o $@

clean:
rm -rf $(BUILD_DIR) $(BIN_DIR)

test: all
@echo 'Running tests...'
./$(TARGET) test_input.hex
```

## 8.6 Day 5 Exercises

### Day 5 Hands-On Exercises

- ▶ Exercise 1: Write a Makefile for a 3-file C project (main.c, utils.c, utils.h) with all, clean, and test targets. Use pattern rules for .o files.
- ▶ Exercise 2: Add a help target that prints all available targets with descriptions.



- Exercise 3: Create a Makefile that uses variables for compiler flags, and allows overriding from command line (e.g., make CFLAGS='-O2').



## 9. Grand Assignment: Module 1 Capstone

### GRAND ASSIGNMENT

MEDS Module 1 — Due: 48 Hours After Day 5

Submission: GitHub repository link to be sent by your mentor

### 9.1 Project Description

You will create a project called `riscv-log-analyzer` — a shell-based tool that processes RISC-V simulation log files, extracts useful information, and generates summary reports. This project ties together every skill from Module 1: Linux commands, shell scripting, Git workflows, and Makefiles.

### 9.2 Repository Structure

Your repository must follow this exact structure:

```
riscv-log-analyzer/
├── README.md           # Project description, usage, examples
├── Makefile            # Build & run automation
├── .gitignore          # Proper ignore rules
├── scripts/
│   ├── analyze.sh      # Main analysis script
│   ├── setup_env.sh    # Environment setup script
│   └── generate_report.sh # Report generation
├── test_data/
│   ├── sample_sim.log  # Sample simulation log
│   ├── sample_pass.log # Log with all tests passing
│   └── sample_fail.log  # Log with some failures
├── output/             # Generated reports (gitignored)
└── docs/
    └── USAGE.md        # Detailed usage guide
```

### 9.3 Requirements

#### Part A: Shell Scripts (40 points)

`analyze.sh` must accept the following arguments:

- `$1`: Path to log file (required)





- `--format [text|csv]`: Output format (default: text)
- `--output <path>`: Output file path (default: stdout)
- `--verbose`: Enable verbose output
- `--help`: Print usage information

The script must analyze the log file and extract:

12. Total number of tests run
13. Number of PASS, FAIL, and SKIP results
14. Pass rate as a percentage
15. List of all failing test names
16. Execution time per test (if available in log)
17. Summary statistics (min/max/avg execution time)

Requirements:

- Must use `set -euo pipefail`
- Must include proper error handling (invalid file, missing arguments)
- Must use at least 2 functions
- Must include inline comments explaining non-obvious logic
- Exit code 0 if all tests pass, 1 if any fail

## Part B: Git Workflow (30 points)

Your Git history must demonstrate proper workflow:

18. Minimum 10 meaningful commits (not “wip” or “asdf”)
19. At least 2 feature branches merged into main
20. At least 1 merge conflict resolved (create one intentionally and resolve it)
21. Proper `.gitignore` (output files, OS files, editor files must be ignored)
22. Clean git log `--oneline --graph --all` showing branch/merge history

## Part C: Makefile (20 points)

Write a Makefile with these targets:

- `all`: Run the analyzer on all test log files
- `test`: Run the analyzer on each `test_data` file and verify expected output
- `report`: Generate a summary report in output/
- `clean`: Remove all generated output files
- `help`: Print all available targets with descriptions
- `setup`: Check that all required tools (bash, grep, awk, etc.) are installed



## Part D: Documentation (10 points)

- README.md with: project title, description, installation, usage with examples, sample output
- USAGE.md with detailed command reference
- Inline comments in all scripts
- Commit messages that tell the story of how you built the project

## 9.4 Sample Log Format

Your scripts should be able to parse log files in this format:

```
[2026-05-01 10:23:45] TEST START: rv32i-add
[2026-05-01 10:23:46] TEST PASS: rv32i-add (0.82s)
[2026-05-01 10:23:46] TEST START: rv32i-sub
[2026-05-01 10:23:47] TEST PASS: rv32i-sub (0.65s)
[2026-05-01 10:23:47] TEST START: rv32i-sll
[2026-05-01 10:23:48] TEST FAIL: rv32i-sll (1.02s)
[2026-05-01 10:23:48] ERROR: Signature mismatch at line 42
[2026-05-01 10:23:48] TEST START: rv32i-srl
[2026-05-01 10:23:48] TEST SKIP: rv32i-srl (not supported)
...
[2026-05-01 10:30:12] SUMMARY: 25 tests, 22 passed, 2 failed, 1 skipped
```

## 9.5 Expected Output Example

```
=== RISC-V Simulation Log Analysis ===
Log file: test_data/sample_fail.log
Analysis date: 2026-05-05 14:30:00

--- Results Summary ---
Total tests: 25
Passed:      22 (88.0%)
Failed:      2 ( 8.0%)
Skipped:     1 ( 4.0%)

--- Failed Tests ---
1. rv32i-sll
2. rv32i-beq

--- Timing Statistics ---
Min time: 0.42s (rv32i-nop)
```



```
Max time: 2.31s (rv32i-mul)
```

```
Avg time: 0.87s
```

```
--- Verdict: FAIL ---
```

```
Exit code: 1
```

## 9.6 Bonus Challenges (+10 points)

- Add a `--compare` flag that diffs two log files and shows regressions (tests that passed before but fail now)
- Add colored output (green for PASS, red for FAIL) using ANSI escape codes
- Write a `generate_report.sh` that outputs an HTML report with a table of results
- Use `getopts` for proper argument parsing instead of manual parsing



## 10. Assessment Rubric

| Component     | Points        | Criteria                                                                                           |
|---------------|---------------|----------------------------------------------------------------------------------------------------|
| Shell Scripts | 40            | Correct parsing, argument handling, error handling, functions, exit codes, code clarity            |
| Git Workflow  | 30            | 10+ meaningful commits, 2+ branches merged, 1+ conflict resolved, clean history, proper .gitignore |
| Makefile      | 20            | All required targets work, proper variables, pattern rules used, help target                       |
| Documentation | 10            | README with usage + examples, USAGE.md, inline comments, meaningful commit messages                |
| Bonus         | +10           | Compare flag, colored output, HTML report, getopts                                                 |
| <b>TOTAL</b>  | <b>100+10</b> |                                                                                                    |

**Grading:** 90–100: Exceptional | 75–89: Proficient | 60–74: Developing | Below 60: Needs Rework

**Policy:** Assignments below 60 must be resubmitted within 24 hours. Two failed resubmissions trigger a conversation with your mentor and manager.



## 11. Recommended Resources

---

### Primary References

- MIT Missing Semester: <https://missing.csail.mit.edu/> — Lectures 1–6 are directly relevant
- Pro Git Book (free): <https://git-scm.com/book/en/v2> — Chapters 1–3, 6–7
- Bash Reference Manual: <https://www.gnu.org/software/bash/manual/>
- GNU Make Manual: <https://www.gnu.org/software/make/manual/>

### Video Resources

- MIT Missing Semester YouTube playlist (same content as website, video format)
- Git for Professionals — freeCodeCamp (YouTube, 40 min)
- Fireship: Git Explained in 100 Seconds (quick conceptual overview)

### Practice Platforms

- Learn Git Branching: <https://learngitbranching.js.org/> — Interactive visual Git tutorial
- OverTheWire Bandit: <https://overthewire.org/wargames/bandit/> — Linux CLI challenges
- Exercism Bash Track: <https://exercism.org/tracks/bash> — Shell scripting exercises

### Quick References

- Git cheat sheet: <https://education.github.com/git-cheat-sheet-education.pdf>
- Bash cheat sheet: <https://devhints.io/bash>
- Vim survival guide: vimtutor (run this command in your terminal)



## 12. Appendix: Quick Reference Cheat Sheets

### A. Essential Linux Commands

```
# Navigation
pwd cd ls tree find which file

# File Operations
cp mv rm mkdir touch ln chmod chown

# Text Processing
cat head tail grep sed awk sort uniq cut wc diff

# Processes
ps aux top/htop kill jobs bg fg Ctrl+C Ctrl+Z

# System Info
df -h du -sh free -h uname -a hostname

# Networking
ssh scp curl wget ping
```

### B. Git Commands Quick Reference

```
# Setup & Init
git init git clone git config

# Daily Workflow
git status git add git commit git push git pull

# Branching
git branch git checkout/switch git merge git rebase

# Inspection
git log git diff git show git blame git reflog

# Undoing
git restore git reset git revert git stash

# Remote
```



```
git remote  git fetch  git pull  git push
```

## C. Makefile Quick Reference

```
# Structure
target: prerequisites
recipe (TAB-indented!)

# Variables
CC = gcc
$(CC)  $@  $<  $^  $*

# Pattern Rules
%.o: %.c
$(CC) -c $< -o $@

# Phony Targets
.PHONY: all clean test
```

---

### End of Module 1

MEDS • UET Lahore • Summer 2026

*"Your terminal is your workshop. Master it."*